Capstone Project “Minstrel – Music Library”

1. Overview:

The **Minstrel -Music Library** is a full-stack web application developed using the **MERN Stack**: **MongoDB, Express.js, React.js, and Node.js**. It serves as an entertainment platform where users can browse, search, and play songs, as well as create and manage playlists.

The system supports two types of users:

- **Admin**
- **User**

The **Admin** manages the music library by adding, updating, and deleting songs and artists.
The **User** can browse songs, search for music, create playlists, and play songs using the built-in music player.

The application is developed as a **Single Page Application (SPA)** using **React Router** and communicates with backend services through **Axios**.

MongoDB is used to store application data such as users, songs, artists, albums, and playlists.

2. Objective:

The main objective of this project is to build a full-stack **Music Library Application** with authentication, database integration, and role-based access control.

Main Objectives

- Implement user registration and login
- Implement admin login
- Perform CRUD operations on songs
- Perform CRUD operations on playlists
- Implement search functionality
- Integrate frontend with backend APIs
- Use MongoDB for database storage
- Use JWT-based authentication
- Build a responsive user interface
- Develop the application as a Single Page Application

3. Technologies Used

Frontend

- React.js
- React Router
- Axios
- CSS

Backend

- Node.js
- Express.js

Database

- MongoDB
- Mongoose

Authentication

- JWT (JSON Web Token)
- bcrypt

Tools and Utilities

- MongoDB Compass
- Visual Studio Code
- GitHub
- Postman

4. System Architecture

The project follows the standard **MERN architecture**.

Application Flow

User → React Frontend → API Request → Node.js/Express.js → MongoDB

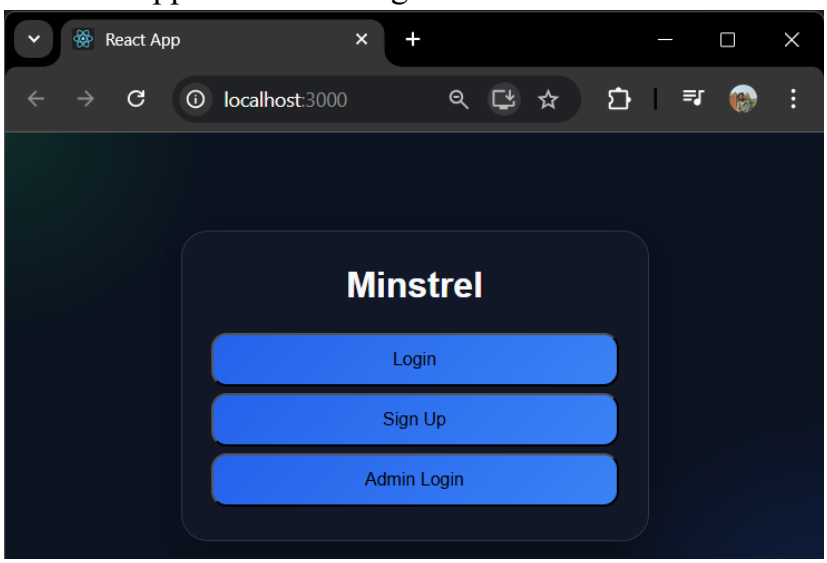
Working Process

1. The user interacts with the React-based user interface.
2. The frontend sends requests to the backend using Axios.
3. Express handles incoming API requests.
4. Controllers process the business logic.
5. MongoDB stores or retrieves the required data.

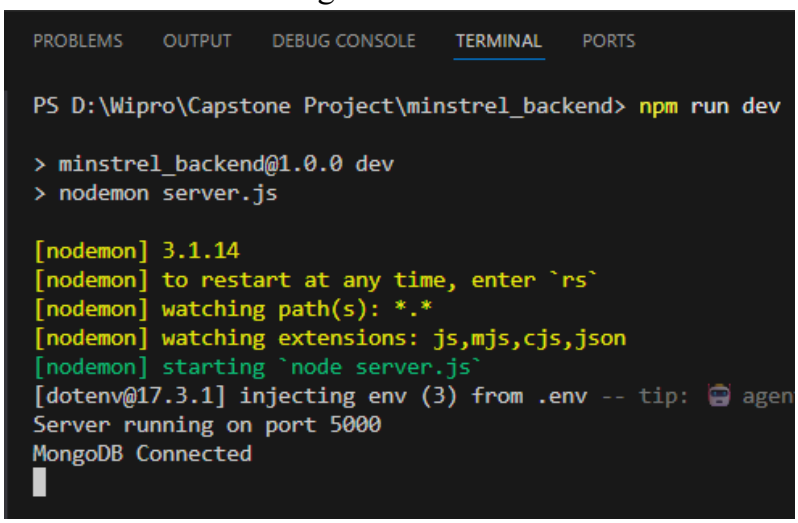
6. The response is returned to the frontend and displayed to the user.

Screenshot:

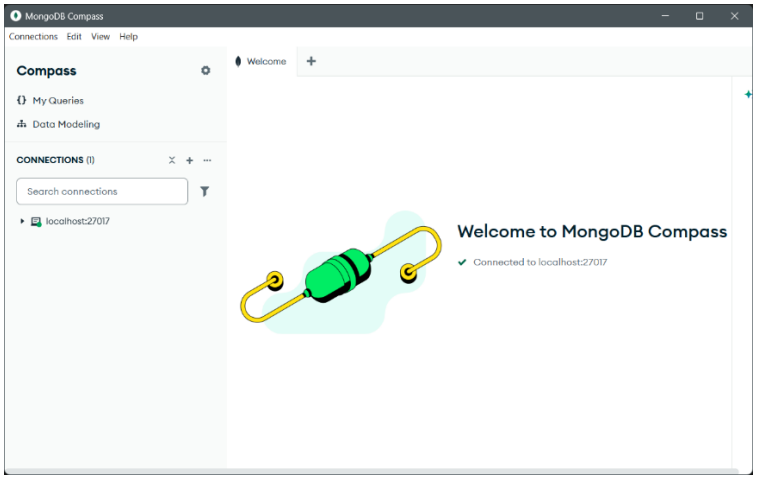
Frontend application running in browser



Backend server running in terminal



MongoDB Compass connected



5. Database Schema:

The application uses the following MongoDB collections:

- **User**
- **Song**
- **Artist**
- **Album**
- **Playlist**

User Collection Fields

- username
- email
- password
- likedSongs
- playlists

Song Collection Fields

- title
- artist
- album
- audioUrl
- duration
- createdAt

Artist Collection Fields

- name
- image

Playlist Collection Fields

- name
- songs
- userId

Screenshot (Schema):

minstrel +

localhost:27017 > minstrel

Open MongoDB shell | Create collection | Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
albums	-	36.86 kB	462.00 B	5	92.00 B	1	36.86 kB
artists	-	36.86 kB	441.00 B	6	73.00 B	1	36.86 kB
playlists	-	4.10 kB	0 B	0	0 B	1	4.10 kB
songs	-	32.77 kB	224.00 B	1	224.00 B	1	36.86 kB
users	-	20.48 kB	209.00 B	1	209.00 B	4	81.92 kB

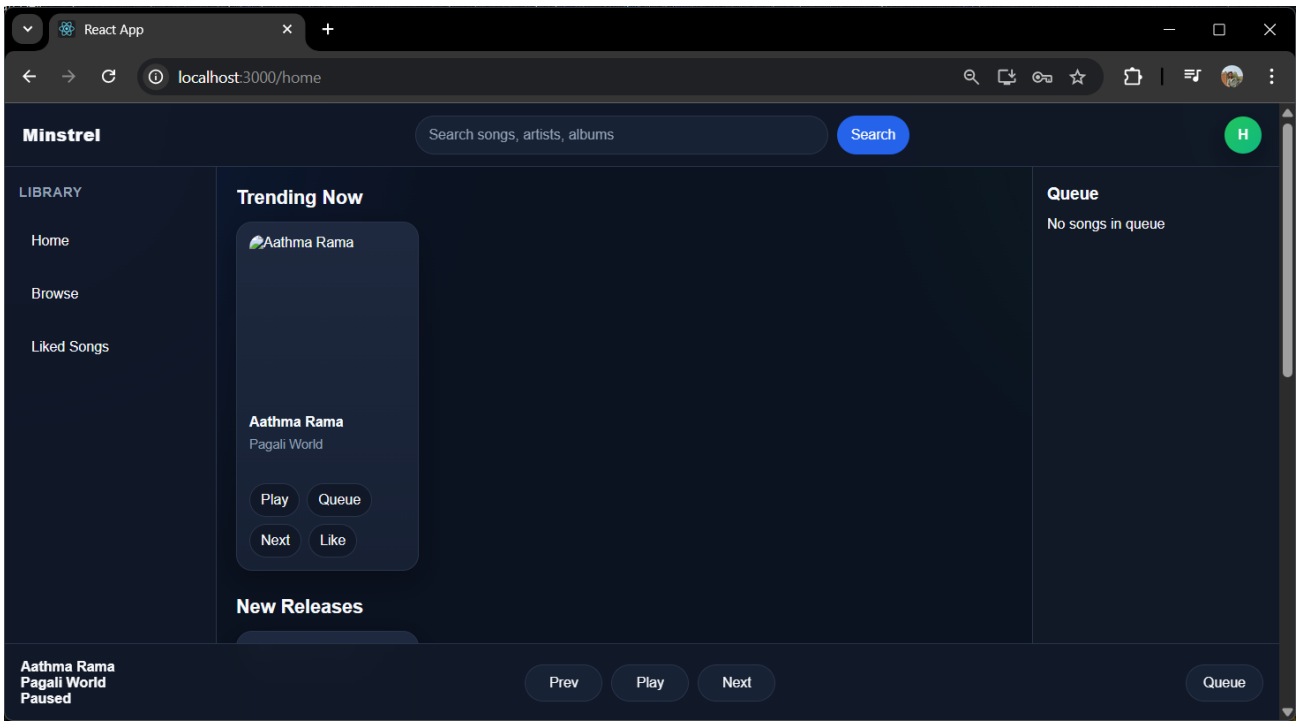
6. User Features

The application provides the following features for normal users:

- Register an account
- Login and logout
- View songs
- Search songs
- View artist pages
- View album pages
- Create playlists
- Delete playlists
- Add songs to playlists
- Play songs
- Pause songs
- Shuffle songs
- Repeat songs
- Manage playback queue

Screenshot

User Home page



7. Admin Features

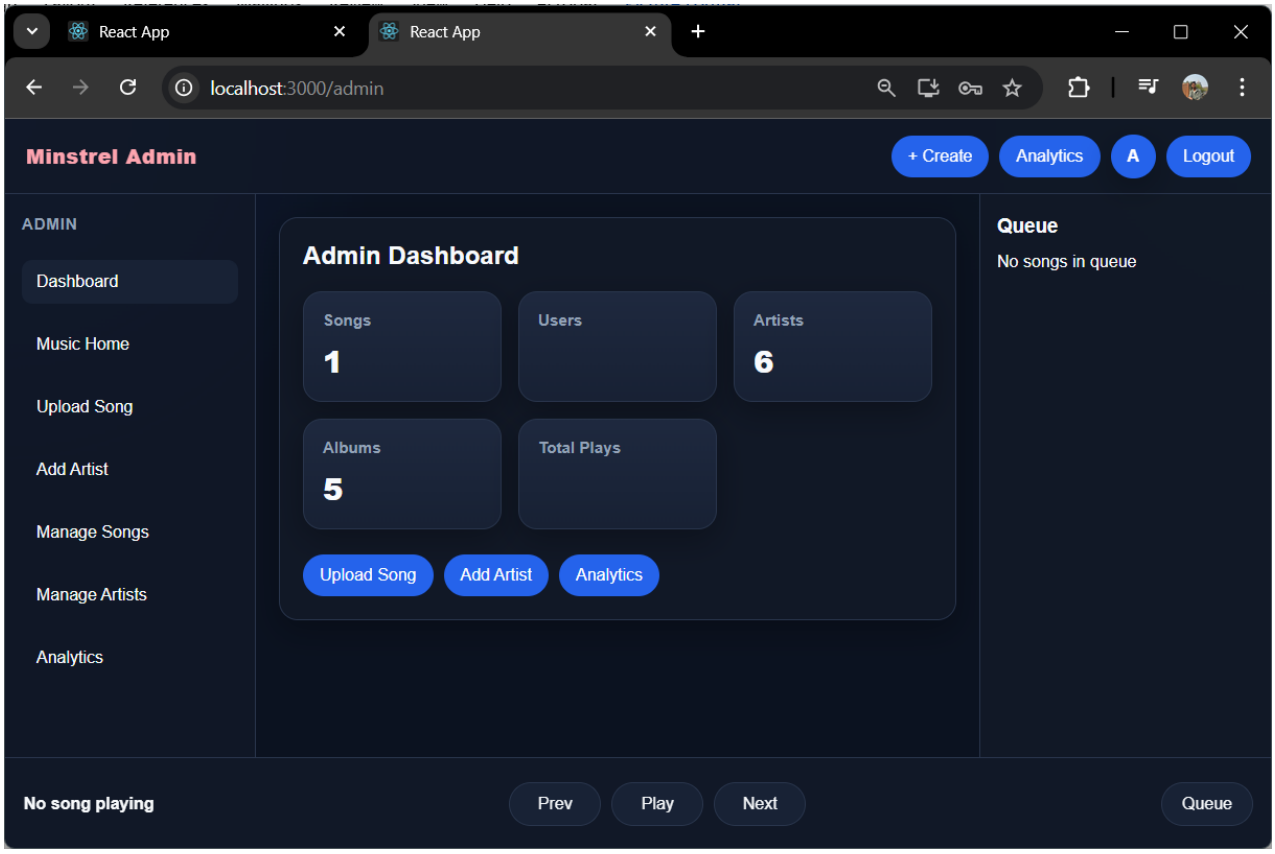
The admin module provides management and monitoring functionalities for the platform.

Admin Functionalities

- Admin login
- Add song
- Add artist
- Delete song
- View statistics
- Manage songs
- Manage artists
- Upload multiple songs
- View analytics

Screenshot:

Admin dashboard



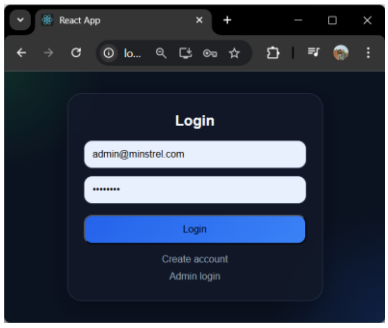
8. Authentication

Authentication is implemented using **JWT (JSON Web Token)**.

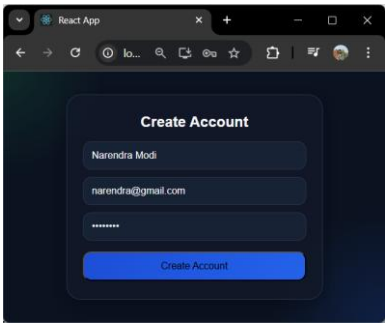
Authentication Process

1. The user registers an account.
2. The password is securely hashed using **bcrypt**.
3. The user logs in using valid credentials.
4. A JWT token is generated after successful login.
5. The token is stored in **localStorage**.
6. The token is sent along with protected API requests.
7. Protected routes verify the token before granting access.
8. Admin access is restricted based on user role.

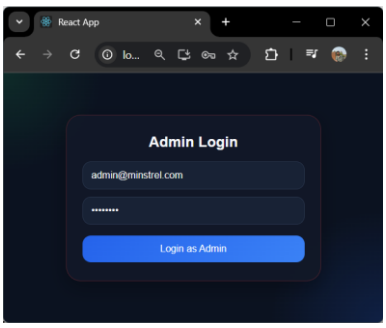
Screenshot



User login page



Signup page



Admin login page

9. API Structure

The backend exposes multiple APIs for handling authentication, songs, artists, playlists, and admin operations.

Authentication APIs

- POST /auth/signup
- POST /auth/login

Song APIs

- GET /songs
- POST /admin/upload-song
- DELETE /admin/songs/:id

Artist APIs

- GET /artists
- POST /admin/artists

Playlist APIs

- POST /playlist
- GET /playlist
- DELETE /playlist

Admin APIs

- GET /admin/stats

10. Testing

Testing was carried out using multiple methods to ensure the proper functioning of the application.

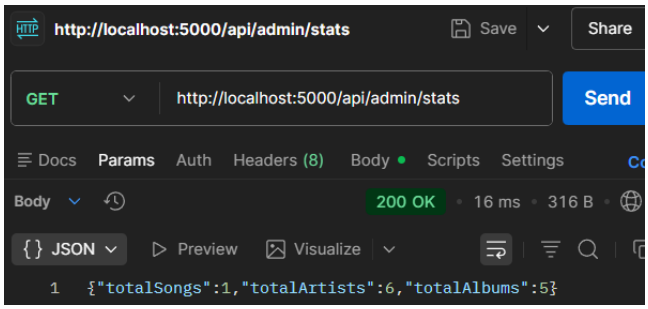
Tools Used for Testing

- Postman
- Browser testing
- MongoDB Compass

Tested Modules

- User login
- User signup
- Add song
- Delete song
- Playlist CRUD operations
- Search functionality
- Admin statistics

Screenshot



11. Future Scope

The application can be further improved by adding advanced features such as:

- Push notifications
- Song recommendation system
- Cloud-based audio storage
- Dark/light theme preference saved in database
- Mobile-responsive enhanced version (with Layout Change)
- Enhanced Admin Dashboard

12. Conclusion

The **Minstrel Music Library Application** successfully demonstrates a complete full-stack implementation using the **MERN Stack**.

The project includes authentication, CRUD operations, role-based access control, database integration, API communication, and a responsive user interface.

Overall, the application meets both **user-side** and **admin-side** requirements and reflects a practical implementation of modern web application development.